

iOS 项目网络请求实现分析与优化

一、网络请求架构解析

项目采用自定义网络请求框架 `MKNetworkModel`，基于 `MKNetworkEngineEx` 封装，整体架构清晰，提供多类型请求、缓存、图片上传等核心功能，满足基础业务需求。

1. 基础架构（初始化与 URL 处理）

核心逻辑为初始化网络引擎实例，并统一 URL 格式（移除冗余前缀 / 后缀），避免格式不一致导致的请求失败。

```
// 初始化方法，设置基础 URL 与 API 路径
- (id)initWithURL:(NSString *)url apiPath:(NSString *)apiPath {
    if (self = [super init]) {
        // 处理 URL 前缀（移除 "http://"，统一格式）
        NSString *pre = @"http://";
        if ([url hasPrefix:pre]) {
            url = [url stringByReplacingOccurrencesOfString:pre withString:@""];
        }
        // 处理 URL 后缀（移除末尾 "/"，避免拼接路径时出现重复 "/"）
        if ([url hasSuffix:@"/"]) {
            url = [url substringToIndex:url.length - 1];
        }
        // 创建网络请求引擎实例（核心依赖 MKNetworkEngineEx）
        _netWorkEngineEx = [[MKNetworkEngineEx alloc] initWithHostName:url
                                                                    apiPath:apiPath
                                                                    customHeaderFields:nil];
    }
    return self;
}
```

2. 四种请求类型（按返回数据格式划分）

框架针对不同业务场景，提供四种返回类型的请求方法，覆盖 “无数据返回” “列表数据” “详情数据” “模型数据” 等常见需求：

请求类型	适用场景	核心特点
void 类型	仅需判断成功 / 失败（如表单提交）	无返回数据，仅回调成功 / 失败状态
数组类型	获取列表数据（如商品列表、消息列表）	返回 NSArray，可直接绑定列表控件
字典类型	获取单条数据详情（如用户信息）	返回 NSDictionary，灵活解析字段
Model 类型	复杂数据结构（需映射为模型）	基于 Mantle 库自动将 JSON 映射为 Model

字典类型请求示例代码（核心逻辑含加载指示器、缓存、回调处理）：

```
- (MKNetworkOperationEx *)requestDictionaryReturnWithParameter:(NSDictionary *)dic
    onKeyPath:(NSString *)keyPath
    onSuccessed:(DictionaryBlock)succeededBlock
    onError:(ErrorBlock)errorBlock
    WithHttpMethod:(NSString *)httpMethod
    withSSL:(BOOL)isSSL
    withProgress:(UIView *)progressContainerView
    progressText:(NSString *)progressText {
    // 1. 显示加载指示器 (MBProgressHUD)
    if (progressContainerView) {
        MBProgressHUD *hud = [MBProgressHUD showHUDAddedTo:progressContainerView
        animated:YES];
        if (progressText) {
            hud.labelText = progressText; // 自定义加载提示文字
        }
    }
    // 2. 定义成功回调（弱引用避免循环引用）
    DEFINE_WEAK(self);
    DictionaryBlock OnSucceededBlock = ^(NSMutableDictionary *results) {
        // 隐藏加载指示器
        if (progressContainerView) {
            [MBProgressHUD hideHUDForView:progressContainerView animated:YES];
        }
    };
```

```

    }
    // 启用缓存时保存请求结果
    if (wself.bEnableSaveCahce) {
        [wself saveReciveCookie:results params:dic];
    }
    // 调用业务层传入的成功回调
    if (succeededBlock) {
        succeededBlock(results);
    }
};

// 3. 定义错误回调（逻辑类似成功回调，处理错误信息后回调业务层）
NSErrorBlock OnErrorBlock = ^(NSError *error) {
    if (progressContainerView) {
        [MBProgressHUD hideHUDForView:progressContainerView animated:YES];
    }
    if (errorBlock) {
        errorBlock(error);
    }
};

// 4. 创建并执行网络请求
_operation = [_netWorkEngineEx requestDictionaryReturnWithParameter:dic
                                onKeyPath:keyPath
                                onSuccessed:OnSucceededBlock
                                onError:OnErrorBlock
                                WithHttpMethod:httpMethod
                                withSSL:isSSL];
_operation.params = dic; // 保存请求参数，便于后续追踪
return _operation;
}

```

3. 缓存机制（基于 TMCache 实现）

核心逻辑：以“URL + 合并参数”的 MD5 哈希值为缓存键，存储请求结果，减少重复请求，提升性能。

```

- (void)saveReciveCookie:(id)data params:(NSDictionary *)params {
    // 1. 合并基础参数（如设备 ID、Token）与业务参数
    NSMutableDictionary *baseParams = [MKNetworkOperationEx baseParams];

```

```

[baseParams addEntriesFromDictionary:params];
// 2. 构建缓存键（URL + 参数拼接后 MD5 哈希，避免键过长）
if (_netWorkEngineEx.exUrl) {
    __block NSString *url = [NSString stringWithFormat:@"%s?", _netWorkEngineEx.exUrl];
    // 移除空值或无效参数，避免缓存键冗余
    NSMutableDictionary *postDict = [MKNetworkOperationEx deleteMutableParamsInPostDictionary:baseParams];
    // 拼接参数到 URL
    [postDict enumerateKeysAndObjectsUsingBlock:^(NSString *key, id object, BOOL *stop) {
        url = [url stringByAppendingFormat:@"%s=%s", key, object];
    }];
    // 3. 存储缓存（TMCache 为第三方缓存库，支持内存+磁盘双缓存）
    [[TMCache sharedCache] setObject:data forKey:[url MD5Hash]];
}
}

```

4. 图片上传功能（支持进度回调）

提供文件上传接口，支持设置 MIME 类型、上传进度监听，且支持“可冻结”（网络中断后恢复上传），适配弱网场景。

```

- (MKNetworkOperationEx *)uploadParams:(NSMutableDictionary *)params
    file:(NSString *)file
    key:(NSString *)key
    mimeType:(NSString *)fileType
    progress:(MKNetworkProgressBlock)progressBlock {
    // 省略参数校验与路径处理逻辑...

    // 1. 创建 POST 上传操作
    MKNetworkOperationEx *op = (MKNetworkOperationEx *)[_netWorkEngineEx
        operationWithPathEx:action
                        params:params
                httpMethod:@"POST"];

    // 2. 添加上传文件（文件路径、表单键、MIME 类型）
    [op addFile:file forKey:key mimeType:fileType];

    // 3. 配置上传特性（支持网络恢复后继续上传）
    [op setFreezable:YES];
}

```

```
// 4. 设置上传进度回调 (0.0~1.0, 用于更新 UI 进度条)
[op onUploadProgressChanged:progressBlock];

// 5. 入队执行上传任务
[_netWorkEngineEx enqueueOperation:op];

return op;
}
```

二、代码优化建议

当前框架实现了基础功能，但在“现代性、健壮性、可维护性”方面存在提升空间，具体优化方案如下：

1. 升级至现代网络框架（替代 MKNetworkEngine）

问题： MKNetworkEngine 为老旧框架，不支持 URLSession 新特性（如后台下载、流处理、证书验证），且社区维护停滞，存在兼容性风险。

优化方案： 基于系统 URLSession 或第三方库 Alamofire 重写，示例如下（基于 URLSession）：

```
// 基于 URLSession 的通用请求方法
- (void)requestWithURL:(NSString *)url
    method:(NSString *)method
    params:(NSDictionary *)params
    success:(void (^)(NSDictionary *))success
    failure:(void (^)(NSError *))failure {
    // 1. 校验 URL 合法性
    NSURL *requestURL = [NSURL URLWithString:url];
    if (!requestURL) {
        if (failure) failure([NSError errorWithDomain:@"InvalidURL" code:-1 userInfo:@{
        NSLocalizedDescriptionKey:@"URL 格式非法"}]);
        return;
    }

    // 2. 创建请求对象
    NSMutableURLRequest *request = [NSMutableURLRequest requestWithURL:requestURL];
    request.HTTPMethod = method; // GET/POST/PUT/DELETE
```

```

request.timeoutInterval = 30.0; // 设置超时时间（30秒）

// 3. 设置请求头（如 JSON 格式）
[request setValue:@"application/json" forHTTPHeaderField:@"Content-Type"];
[request setValue:@"iOS" forHTTPHeaderField:@"User-Agent"]; // 自定义 User-Agent
// 4. 处理 POST/PUT 等请求的参数（转为 JSON 数据）
if ([method isEqualToString:@"POST"] || [method isEqualToString:@"PUT"]) && params) {
    NSError *jsonError;
    NSData *jsonData = [NSJSONSerialization dataWithJSONObject:params
                                                options:0
                                                error:&jsonError];

    if (jsonError) {
        if (failure) failure(jsonError);
        return;
    }
    request.HTTPBody = jsonData;
}
// 5. 创建会话任务（异步执行）
NSURLSessionDataTask *task = [[NSURLSession sharedSession] dataTaskWithRequest:
request
                                completionHandler:^(NSData * _Nullable data,
                                NSURLResponse * _Nullable response,
                                NSError * _Nullable error) {
// 6. 主线程回调（UI 操作需在主线程）
dispatch_async(dispatch_get_main_queue(), ^{
    if (error) {
        if (failure) failure(error);
        return;
    }
// 7. 解析 JSON 响应（校验响应格式）
    NSError *parseError;
    NSDictionary *result = [NSJSONSerialization JSONObjectWithData:data
                                                                options:0
                                                                error:&parseError];

    if (parseError) {
        if (failure) failure(parseError);
        return;
    }
    if (success) success(result);
});
}];
}];

```

```
// 8. 启动任务
[task resume];
}
```

2. 统一错误处理（分类处理网络异常）

问题：错误处理逻辑分散在各请求方法中，提示信息笼统（如仅“请求失败”），未区分“网络错误”“服务器错误”“解析错误”，用户体验差。

优化方案：创建统一错误处理工具类，根据错误类型与错误码返回具体提示。

```
// 统一错误处理工具类（NetworkErrorHandler.h）
@interface NetworkErrorHandler : NSObject
+ (NSError *)handleNetworkError:(NSError *)error; // 处理网络相关错误
+ (NSError *)handleParseError:(NSError *)error; // 处理 JSON 解析错误
@end

// 实现（NetworkErrorHandler.m）
@implementation NetworkErrorHandler
+ (NSError *)handleNetworkError:(NSError *)error {
    NSMutableDictionary *userInfo = [error.userInfo mutableCopy];
    NSString *errorMsg = @"网络请求失败，请稍后重试";
    // 根据错误码细化提示（覆盖常见网络错误）
    switch (error.code) {
        case NSURLErrorNotConnectedToInternet:
            errorMsg = @"网络未连接，请检查 WiFi 或蜂窝网络";
            break;
        case NSURLErrorTimedOut:
            errorMsg = @"请求超时，请检查网络稳定性";
            break;
        case NSURLErrorCannotConnectToHost:
            errorMsg = @"服务器连接失败，可能正在维护";
            break;
        case NSURLErrorNetworkConnectionLost:
            errorMsg = @"网络连接中断，请重新尝试";
            break;
        case NSURLErrorBadServerResponse:
            errorMsg = @"服务器响应异常，请稍后重试";
            break;
        default:
            break;
    }
    return [NSError errorWithDomain:@"com.example.network" code:-1 userInfo:userInfo];
}
```

```

    }
    // 更新错误信息（便于业务层获取具体提示）
    userInfo[NSLocalizedStringKey] = errorMsg;
    return [[NSError alloc] initWithDomain:error.domain
        code:error.code
        userInfo:userInfo];
}
+ (NSError *)handleParseError:(NSError *)error {
    NSMutableDictionary *userInfo = [error.userInfo mutableCopy];
    userInfo[NSLocalizedStringKey] = @"数据解析失败，请联系客服";
    return [[NSError alloc] initWithDomain:@"ParseError"
        code:-2
        userInfo:userInfo];
}
@end
// 使用示例（请求回调中）
if (error) {
    // 判断错误类型并处理
    if ([error.domain isEqualToString:NSURLErrorDomain]) {
        error = [NetworkErrorHandler handleNetworkError:error];
    } else if ([error.domain isEqualToString:@"ParseError"]) {
        error = [NetworkErrorHandler handleParseError:error];
    }
    if (failure) failure(error);
    return;
}

```

3. 缓存机制优化（添加过期策略）

问题：当前缓存无过期时间，可能显示陈旧数据（如缓存的用户信息、商品价格未更新），导致业务逻辑异常。

优化方案：存储缓存时记录时间戳，读取时检查是否过期，支持自定义过期时间。

```

// 带过期时间的缓存管理类（CacheManager.h）
@interface CacheManager : NSObject
+ (instancetype)sharedManager;
// 存储缓存（expiration：过期时间，单位秒；0 表示永不过期）
- (void)saveObject:(id)object forKey:(NSString *)key expiration:(NSTimeInterval)expiration;
// 读取缓存（自动检查是否过期，过期返回 nil）

```



```

- (id)objectForKey:(NSString *)key;
// 手动移除指定缓存
- (void)removeObjectForKey:(NSString *)key;
// 清空所有缓存
- (void)clearAllCache;
@end
// 实现 (CacheManager.m)
@implementation CacheManager
+ (instancetype)sharedManager {
    static CacheManager *instance;
    static dispatch_once_t onceToken;
    dispatch_once(&onceToken, ^{
        instance = [[CacheManager alloc] init];
    });
    return instance;
}
- (void)saveObject:(id)object forKey:(NSString *)key expiration:(NSTimeInterval)expiration {
    if (!object || !key) return;

    // 存储数据+时间戳 (时间戳用于后续过期判断)
    NSDictionary *cacheItem = @{
        @"data": object,
        @"timestamp": @([[NSDate date] timeIntervalSince1970]) // 当前时间戳 (秒)
    };

    // 使用 TMCache 存储 (支持内存+磁盘缓存)
    [[TMCache sharedCache] setObject:cacheItem forKey:key];
    // 过期自动清理 (若设置过期时间)
    if (expiration > 0) {
        dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(expiration * NSEC_PER_SEC)),
            dispatch_get_main_queue(), ^{
                [self removeObjectForKey:key];
            });
    }
}
- (id)objectForKey:(NSString *)key {
    if (!key) return nil;

    // 从缓存读取数据
    NSDictionary *cacheItem = [[TMCache sharedCache] objectForKey:key];

```

```

if (!cacheItem) return nil;
// 检查是否过期（从业务层获取该 key 的过期时间，如用户信息设置 5 分钟过期）
NSTimeInterval timestamp = [cacheItem[@"timestamp"] doubleValue];
NSTimeInterval currentTime = [[NSDate date] timeIntervalSince1970];
NSTimeInterval expiration = [self getExpirationForKey:key]; // 自定义方法：根据 key 获取过期时间

if (currentTime - timestamp > expiration) {
    [self removeObjectForKey:key]; // 过期则删除缓存
    return nil;
}
return cacheItem[@"data"]; // 返回有效缓存
}

// 自定义方法：根据 key 定义不同过期时间（示例）
- (NSTimeInterval)getExpirationForKey:(NSString *)key {
    if ([key containsString:@"userInfo"]) {
        return 300; // 用户信息：5 分钟过期（300 秒）
    } else if ([key containsString:@"productList"]) {
        return 600; // 商品列表：10 分钟过期（600 秒）
    } else {
        return 3600; // 其他数据：1 小时过期（3600 秒）
    }
}

- (void)removeObjectForKey:(NSString *)key {
    if (key) [[TMCache sharedCache] removeObjectForKey:key];
}

- (void)clearAllCache {
    [[TMCache sharedCache] removeAllObjects];
}

@end

```

4. 添加请求重试机制

（注：文档部分内容可能由 AI 生成）